EXPERIMENT 10: L2 Regularisation, Dropout and Early Stopping Comparison

1. Aim

To investigate overfitting in a Multilayer Perceptron and compare L2 regularisation, dropout and early stopping as methods for improving generalization on a synthetic diabetes-prediction dataset.

2. Mathematical Formulae

Binary cross-entropy loss

$$L_{BCE} = -\frac{1}{m}\sum_{i=1}^{m}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

L2 regularisation

$$L_{total} = L_{BCE} + \lambda \sum_{j} w_j^2$$

Where λ controls the strength of regularisation.

Dropout

During training, a random subset of activations is temporarily set to zero. With inverted dropout, the remaining activations are scaled to preserve their expected value.

Early stopping

Training is stopped when the validation loss fails to improve for a specified number of epochs, called the patience.

Generalization gap

$$Gap = Accuracy_{train} - Accuracy_{test}$$

A smaller positive gap can indicate better generalization, although the gap must be considered alongside the test accuracy.

3. System Architecture



4. Algorithm / Procedure

1. Generate a synthetic dataset containing clinical features and a binary diabetes label.
2. Split the data into training, validation and test sets.
3. Standardize the features using training-set statistics.
4. Train a baseline MLP without regularisation.
5. Train models with L2 regularisation at three values of \(\lambda\).
6. Train models with dropout rates of 0.2, 0.3 and 0.5.
7. Train a model with early stopping based on validation loss.
8. Evaluate each model on the test set.
9. Tabulate test accuracy and the training–test accuracy gap.
10. Plot representative loss curves to compare overfitting.

5. Python Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import accuracy_score

np.random.seed(42)
tf.random.set_seed(42)

# Generate a synthetic clinical-style dataset
X, y = make_classification(
    n_samples=600,
    n_features=10,
    n_informative=6,
    n_redundant=2,
    n_classes=2,
    class_sep=0.8,
    flip_y=0.08,
    random_state=42
)

# Split into training, validation and test sets
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30,
    random_state=42, stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50,
    random_state=42, stratify=y_temp
)

# Scale features without using validation/test statistics
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

```
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

def build_model(l2_value=0.0, dropout_rate=0.0):
  regularizer = (
    tf.keras.regularizers.l2(l2_value)
    if l2_value > 0 else None
  )

  model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train.shape[1],)),

    tf.keras.layers.Dense(
      128, activation="relu",
      kernel_regularizer=regularizer
    ),
    tf.keras.layers.Dense(
      128, activation="relu",
      kernel_regularizer=regularizer
    ),

    tf.keras.layers.Dropout(dropout_rate),

    tf.keras.layers.Dense(
      64, activation="relu",
      kernel_regularizer=regularizer
    ),
    tf.keras.layers.Dense(1, activation="sigmoid")
  ])

  model.compile(
```

```
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
  )

  return model

results = []
histories = {}

def train_and_evaluate(name, l2_value=0.0,
                       dropout_rate=0.0, early_stop=False):
  tf.keras.backend.clear_session()
  tf.random.set_seed(42)

  model = build_model(l2_value, dropout_rate)

  callbacks = []

  if early_stop:
    callbacks.append(
      tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=10,
        restore_best_weights=True
      )
    )

  history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
```

```
        epochs=100,
        batch_size=32,
        callbacks=callbacks,
        verbose=0
    )

    # Evaluate the final training and test predictions
    train_pred = (
        model.predict(X_train, verbose=0).ravel() >= 0.5
    ).astype(int)

    test_pred = (
        model.predict(X_test, verbose=0).ravel() >= 0.5
    ).astype(int)

    train_accuracy = accuracy_score(y_train, train_pred)
    test_accuracy = accuracy_score(y_test, test_pred)

    results.append({
        "Model": name,
        "Epochs": len(history.history["loss"]),
        "Train Accuracy": train_accuracy,
        "Test Accuracy": test_accuracy,
        "Train-Test Gap": train_accuracy - test_accuracy
    })

    histories[name] = history.history

# Baseline model
train_and_evaluate("Baseline")
```

```
# L2 regularisation comparison
for value in [0.001, 0.01, 0.1]:
    train_and_evaluate(
        f"L2 lambda={value}",
        l2_value=value
    )

# Dropout comparison
for rate in [0.2, 0.3, 0.5]:
    train_and_evaluate(
        f"Dropout={rate}",
        dropout_rate=rate
    )

# Early stopping
train_and_evaluate(
    "Early Stopping",
    early_stop=True
)

# Display comparison table
results_df = pd.DataFrame(results)

print("\nRegularisation Comparison")
print(results_df.round(4).to_string(index=False))

# Plot loss curves for representative models
models_to_plot = [
    "Baseline",
    "L2 lambda=0.01",
    "Dropout=0.3",
```

```
    "Early Stopping"
]

plt.figure(figsize=(9, 5))

for name in models_to_plot:
    plt.plot(
        histories[name]["val_loss"],
        label=name
    )

plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.title("Comparison of Regularisation Methods")
plt.legend()
plt.show()

# Plot training vs test accuracy
plt.figure(figsize=(10, 5))

x_positions = np.arange(len(results_df))
width = 0.35

plt.bar(
    x_positions - width / 2,
    results_df["Train Accuracy"],
    width,
    label="Training Accuracy"
)

plt.bar(
```

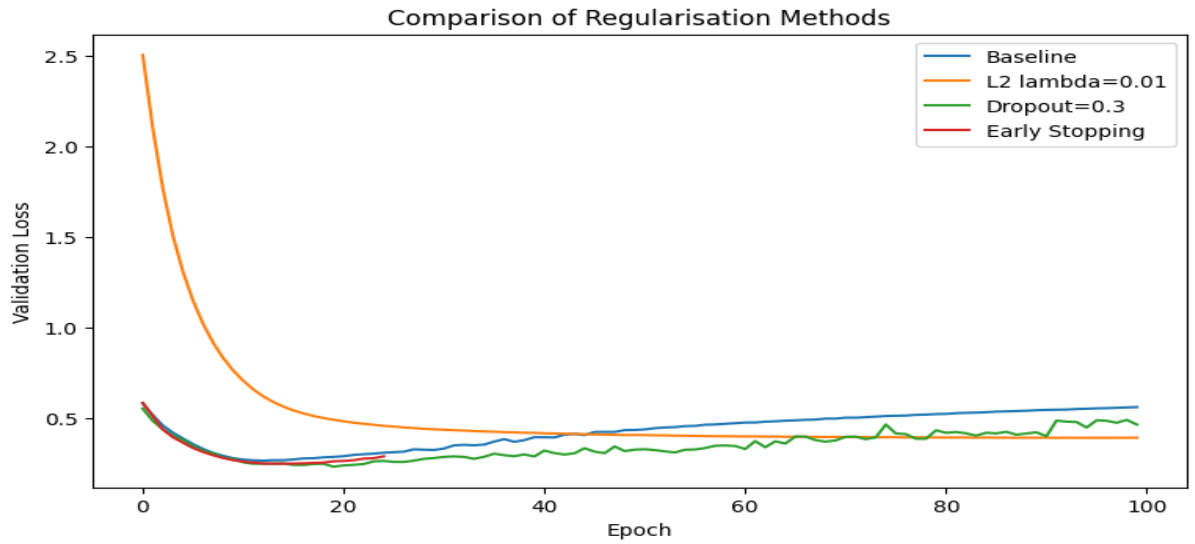
```
    x_positions + width / 2,
    results_df["Test Accuracy"],
    width,
    label="Test Accuracy"
)

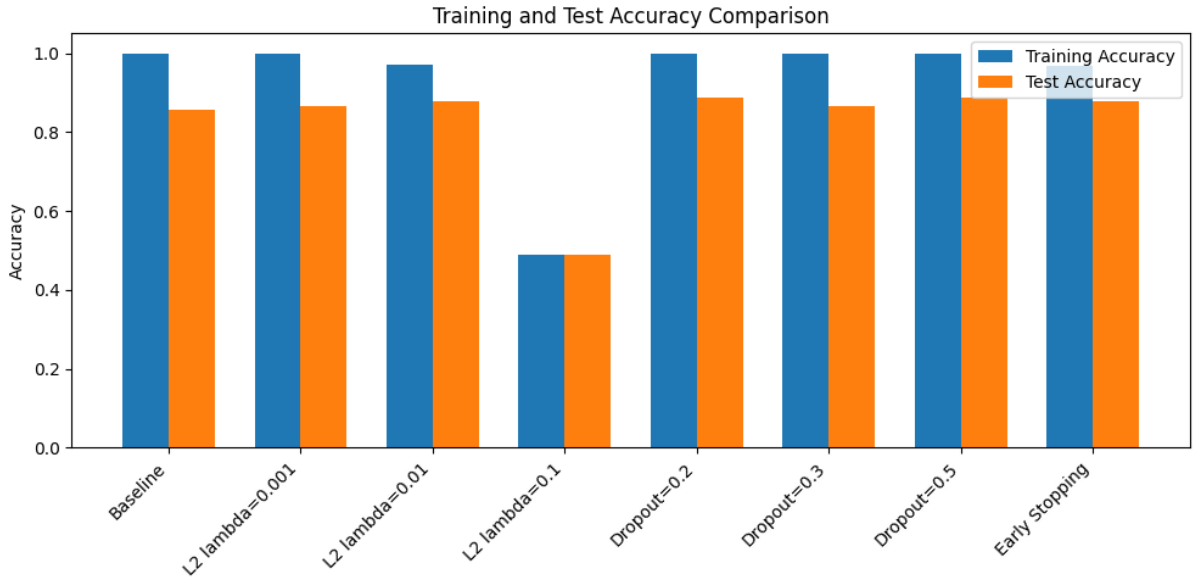
plt.xticks(
    x_positions,
    results_df["Model"],
    rotation=45,
    ha="right"
)

plt.ylabel("Accuracy")
plt.title("Training and Test Accuracy Comparison")
plt.legend()
plt.tight_layout()
plt.show()
```

6. Output

```
Regularisation Comparison
          Model  Epochs  Train Accuracy  Test Accuracy  Train-Test Gap
       Baseline     100          1.0000         0.8556          0.1444
L2 lambda=0.001     100          1.0000         0.8667          0.1333
 L2 lambda=0.01     100          0.9714         0.8778          0.0937
  L2 lambda=0.1     100          0.4905         0.4889          0.0016
    Dropout=0.2     100          1.0000         0.8889          0.1111
    Dropout=0.3     100          1.0000         0.8667          0.1333
    Dropout=0.5     100          1.0000         0.8889          0.1111
 Early Stopping      25          0.9690         0.8778          0.0913
```





7. Result

A baseline MLP and models using L2 regularisation, dropout and early stopping were trained and evaluated. Their test accuracies and train–test gaps were compared to study the effect of regularisation methods on overfitting and generalization.